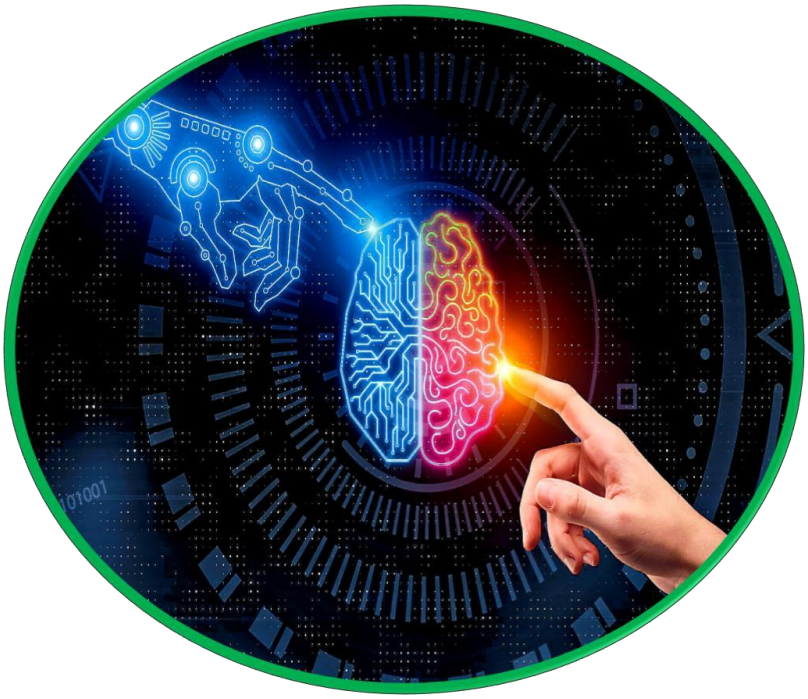
UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL CARRERA DE SOFTWARE
CICLO ACADÉMICO: JULIO – DICIEMBRE 2026

INFORME CLASE N° 2

CLASE CALCULADORA

MANEJO DE PARÁMETROS Y PASO DE VALORES



Integrantes:
Esteban Zambrano
Kevin Garcés
Cristian Gómez
Erick Cordonez
Docente: Ing. José Ruben Caiza Caizabuano

1. Introducción

La programación orientada a objetos (POO) organiza el código modelando objetos del mundo real. Cada objeto tiene datos que lo describen (atributos) y comportamientos que puede realizar (métodos). En Java, esos comportamientos se programan mediante métodos, y para que un mismo método pueda trabajar con distintos datos se utilizan los parámetros.

El presente informe resume la clase N°02, titulada «Manejo de parámetros, paso de valores y clase Calculadora». En él se describen, en el mismo orden de las diapositivas, los conceptos explicados, los ejemplos de código y la práctica realizada con la clase Calculadora y su ejecución en vivo desde la clase Main.

La sesión se centró en practicar la modularidad y el comportamiento de objetos en Java: primero se explicó la diferencia entre parámetros y argumentos, luego el paso por valor y los múltiples parámetros, y finalmente se codificó en vivo en el IDE la clase Calculadora. La sesión dejó al grupo preparado para la siguiente actividad, «Completar código», con la clase Producto.

2. Objetivos

2.1. Objetivo general

Explicar cómo se manejan los parámetros y el paso de valores en los métodos de Java, mediante ejemplos y la construcción de una clase Calculadora que permita practicar la modularidad y la reutilización del código.

2.2. Objetivos específicos

- Diferenciar conceptual y prácticamente un parámetro de un argumento.
- Comprender que en Java los argumentos siempre se pasan por valor y reconocer su efecto sobre las variables originales.
- Definir métodos con múltiples parámetros del mismo tipo o de tipos combinados.
- Estructurar la clase Calculadora con métodos sin retorno (void) y con retorno, incluyendo la validación de la división entre cero.
- Instanciar la clase Calculadora y probar cada uno de sus métodos desde la clase principal.

3. Desarrollo

3.1. Presentación del tema

Se presentó el tema de la clase: Manejo de parámetros, paso de valores y clase Calculadora, bajo el título «Practicando la modularidad y el comportamiento de objetos en Java». Se indicó qué se busca aprender:

- **Modularidad:** organizar el código en clases y métodos.

- **Objetos:** representar elementos del mundo real mediante atributos y comportamientos.
- **Métodos:** definir las acciones que puede realizar cada objeto.
- **Reutilización:** aprovechar el código existente sin tener que escribirlo nuevamente.

3.2. Parámetros vs. argumentos

El objetivo de esta parte fue explicar la diferencia conceptual y práctica entre ambos términos. Un parámetro es la variable declarada en la firma del método y funciona como un espacio reservado para recibir datos. Un argumento es el valor real que se le envía al método cuando se lo llama desde otra parte del código.

Término	Qué es	Dónde aparece
Parámetro	Variable declarada en la firma del método; espacio reservado para recibir datos.	En la definición del método.
Argumento	Valor real que se envía al método cuando se lo llama.	En la llamada al método.

3.3. Ejemplo: método sumar de la calculadora

Ejemplo de código

```
// 'a' y 'b' son PARÁMETROS (Definición del método)
public double sumar(double a, double b) {
    return a + b;
}
```

Cómo se genera (se invoca y se captura el valor)

```
// '15.5' y '4.5' son ARGUMENTOS (Llamada al método)
double resultado = miCalc.sumar(15.5, 4.5);
```

Se llama al método sumar con los argumentos 15.5 y 4.5; estos valores se copian en los parámetros a y b, el método calcula la suma y devuelve 20.0, y ese valor se captura en la variable resultado.

3.4. Paso por valor en Java

Los puntos clave presentados fueron los siguientes:

- En Java, los argumentos SIEMPRE se pasan por valor.
- Cuando se envía una variable a un método, Java entrega una copia de ese valor.
- Modificar la variable del parámetro dentro del método NO afecta la variable original que estaba afuera.

La presentación utilizó un esquema en el que la variable valor1 se copia hacia una variable valor1 dentro del método cambiarValor: son dos variables distintas.

3.5. Ejercicio 1: paso por valor

Enunciado: si se declara int x = 5; y se lo pasa a un método que le suma +10 a su parámetro, ¿cuánto vale x fuera del método después de ejecutarlo?

Respuesta correcta:

```
public void sumarDiez(int n) {
   n = n + 10;   // n vale 15, pero solo dentro del método
}

int x = 5;
sumarDiez(x);
System.out.println(x);   // imprime 5
```

Explicación: x sigue valiendo 5. El método recibe una copia del valor de x; al sumar 10 se modifica solo esa copia (el parámetro n), por lo que la variable original no cambia.

3.6. Múltiples parámetros

Un método puede recibir la cantidad de parámetros que necesite. Estos se separan por comas (,) y pueden ser del mismo tipo de dato o de tipos combinados (String, int, double, boolean).

Ejemplo de código

```
public void registrarOperacion(String tipoOperacion, double resultado, boolean exitosa) {
   System.out.println("Operación: " + tipoOperacion);
   System.out.println("Resultado: " + resultado);
   System.out.println("¿Exitosa?: " + exitosa);
}
```

3.7. Clase Calculadora

El objetivo de esta parte fue mostrar la estructura de la clase práctica que se codificó en vivo en el IDE. La clase combina un método void sin retorno (mostrarMenu) con cuatro métodos con retorno y parámetros (sumar, restar, multiplicar y dividir). El método dividir incluye una validación especial para evitar la división entre cero.

Ejemplo de código

```
public class Calculadora {
   // Método void sin retorno
   public void mostrarMenu() {
```

```
        System.out.println("=== CALCULADORA BÁSICA EN JAVA ===");
        System.out.println("1. Sumar | 2. Restar | 3. Multiplicar | 4. Dividir");
    }

    // Métodos con retorno y parámetros
    public double sumar(double a, double b) {
        return a + b;
    }

    public double restar(double a, double b) {
        return a - b;
    }

    public double multiplicar(double a, double b) {
        return a * b;
    }

    // Validación especial: División entre cero
    public double dividir(double a, double b) {
        if (b == 0) {
            System.out.println("Error: No se puede dividir entre cero.");
            return 0;
        }
        return a / b;
    }
}
```

Método	Retorno	Parámetros	Función
mostrarMenu()	void	Ninguno	Muestra el título y las opciones de la calculadora.
sumar(a, b)	double	double a, double b	Devuelve a + b.
restar(a, b)	double	double a, double b	Devuelve a - b.
multiplicar(a, b)	double	double a, double b	Devuelve a * b.
dividir(a, b)	double	double a, double b	Devuelve a / b; si b es 0 muestra un error y devuelve 0.

3.8. Ejecución en vivo (clase Main)

Esta parte mostró cómo instanciar la clase Calculadora y probar cada uno de sus métodos en la clase principal.

Cómo se genera (se instancia y se invoca)

```
public class Main {
    public static void main(String[] args) {
        // Instanciación del objeto
        Calculadora calc = new Calculadora();

        // Llamada al método void
        calc.mostrarMenu();

        // Invocación de métodos con retorno
        double totalSuma = calc.sumar(10.5, 5.5);
        System.out.println("Resultado Suma: " + totalSuma);

        double totalDiv = calc.dividir(20, 0); // Prueba de la validación
    }
}
```

Se crea el objeto calc de la clase Calculadora, se llama al método void mostrarMenu(), se invoca sumar con los argumentos 10.5 y 5.5 y el valor devuelto (16.0) se captura en totalSuma, que luego se imprime. Por último, se prueba la validación llamando a dividir(20, 0).

3.9. Comparación entre método void y método con return en la calculadora

Característica	Método void	Método con return
¿Devuelve un valor?	No	Sí
Ejemplo en la clase	mostrarMenu()	sumar(double a, double b), dividir(double a, double b)
Tipo de retorno en la firma	void	double
Cómo se invoca	calc.mostrarMenu();	double totalSuma = calc.sumar(10.5, 5.5);

4. Análisis de resultados

Al ejecutar la clase Main se obtiene la siguiente salida en consola:

```
=== CALCULADORA BÁSICA EN JAVA ===
1. Sumar | 2. Restar | 3. Multiplicar | 4. Dividir
Resultado Suma: 16.0
```

Error: No se puede dividir entre cero.

Instrucción	Resultado	Observación
calc.mostrarMenu();	Imprime el menú	Método void: realiza una acción sin devolver valor.
calc.sumar(10.5, 5.5);	16.0	Método con retorno: el valor se captura en totalSuma.
calc.dividir(20, 0);	Mensaje de error y retorno de 0	La validación b == 0 evita la división entre cero.

La ejecución en vivo permitió comprobar que los parámetros hacen posible reutilizar un mismo método con distintos datos, que los métodos con retorno entregan un resultado que otra parte del programa puede guardar y que un método void solo ejecuta una acción. Además, la prueba dividir(20, 0) mostró que los métodos deben ser capaces de validar sus entradas: en lugar de fallar, la calculadora informa el error y devuelve 0.

De esta manera, la explicación de parámetros y argumentos, el paso por valor, los ejemplos de código y la práctica con la clase Calculadora contribuyeron a una mejor comprensión del tema.

5. Conclusiones

- Un parámetro es la variable declarada en la firma del método, mientras que un argumento es el valor real que se envía al llamarlo.
- Flexibilidad: los parámetros permiten que los métodos procesen distintos datos sin reescribir código.
- Seguridad: el paso por valor protege las variables originales de ser modificadas accidentalmente.
- Validación: los métodos deben ser capaces de validar sus entradas, como prevenir divisiones para b == 0.
- Preparación: con lo visto en la clase, el grupo queda listo para la sesión «Completar código» con la clase Producto.

6. Recomendaciones

- Practicar creando métodos con uno o varios parámetros, de tipos iguales y combinados, para afianzar la sintaxis.
- Probar la clase Calculadora en un entorno de desarrollo, cambiando los argumentos para observar cómo varían los resultados.
- Comprobar el paso por valor con ejemplos propios, como el de la variable x, para verificar que la variable original no cambia.
- Incluir validaciones en los métodos que lo requieran, como la división entre cero.

7. Bibliografía

Oracle. (s. f.-a). Defining methods. The Java Tutorials. https://docs.oracle.com/javase/tutorial/java/javaOO/methods.html

Oracle. (s. f.-b). Passing information to a method or a constructor. The Java Tutorials. https://docs.oracle.com/javase/tutorial/java/javaOO/arguments.html

Oracle. (s. f.-c). Returning a value from a method. The Java Tutorials. https://docs.oracle.com/javase/tutorial/java/javaOO/returnvalue.html